

CM/TD 1 — UE3 : BIG DATA, INTRO ET HADOOP



ING 3 : Option Intelligence Artificielle – 2026/2027

Rakib SHEIKH ^{1,2}

rakib.sheikh@cyu.fr

¹ Thales Romania SRL

² CY Cergy Paris Université - CY-Tech



Les intervenants du cours



Rakib SHEIKH

Chargé de cours à CY Cergy Paris Université

Data Scientist - MLOps - Thales Roumanie



Point Essentiels

Déroulé du cours :

- Séparation en séance CM-TD avec des séances de TP.
- Pas de projet —> Examen pratique sous la forme d'un projet, à rendre sous la forme d'un Pull-Request.
- Horaires de cours :
 - *Matiné* : 9h00 – 12h00 (Une pause de minimum 15min est prévu pour contrainte réunion à 10h le lundi et mercredi)
 - *Après Midi* : 13h30 – 17h00

Pré-requis :

- Maîtrise du langage Java (pour utiliser Scala)
- Linux / Mac (Hadoop n'est pas disponible sur Windows en natif, mais peut marcher via WSL)



Outline

Introduction du Big Data (J1-P1)	4
Panorama Eco-système Hadoop (J1-P2)	46

01

Introduction du Big Data (J1-P1)





Historique d'un point de vue Data Marketing

Petit rappel de ce qui s'est passé dans l'histoire des version du Web dans le Data Marketing :

- En 2004, nous avons eu l'apparition de Facebook. (Web 2.0 Réseau Social)
- En 2008 iPhone était présenté comme étant le tout premier smartphone. (Web 3.0)
- A partir des années 2014, nous commençons à voir l'émergence de l'IoT. (Web 4.0)
- Et depuis 2019, nous avons l'association des smartphones, de l'IoT, appliqué à l'IA (Web 5.0)

Un point commun parmi ces points histoire : **La croissance exponentiel du stockage des données.**



Historique : Les bases de données

- À partir des années 70, il y a eu une rupture de l'innovation : C'est la création des bases de données relationnel. (OLTP)
- Dans les années 80, il y a eu l'apparition des bases de données avancées (relationnel étendu, objet...)
- Dans les années 90 :
 - Bases de données Web (et XML)
 - Data Warehouse OLAP, Knowledge Data Discovery, Data Mining

Il y a eu une diversification des types de données que l'on peut stocker dans les BDD, tel que les vidéos (Youtube), Images (Picassa, Facebook), Audio (SoundCloud)

- Fin des années 2000 : Apparition du NoSQL

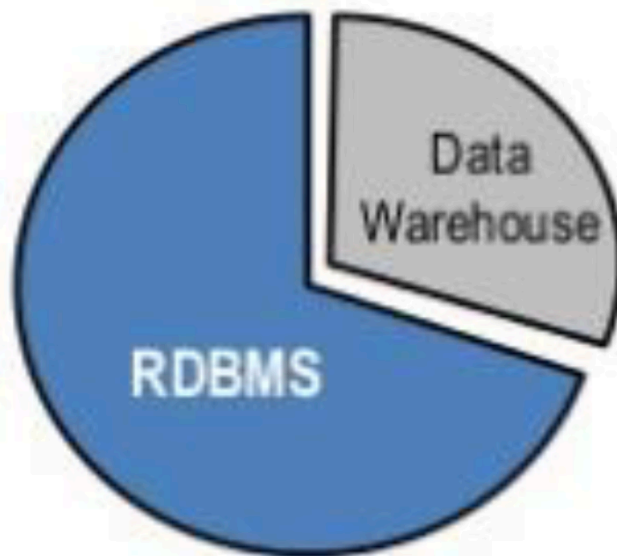
Evidence d'une croissance continue des quantités de données, nous parlons maintenant de Big Data !



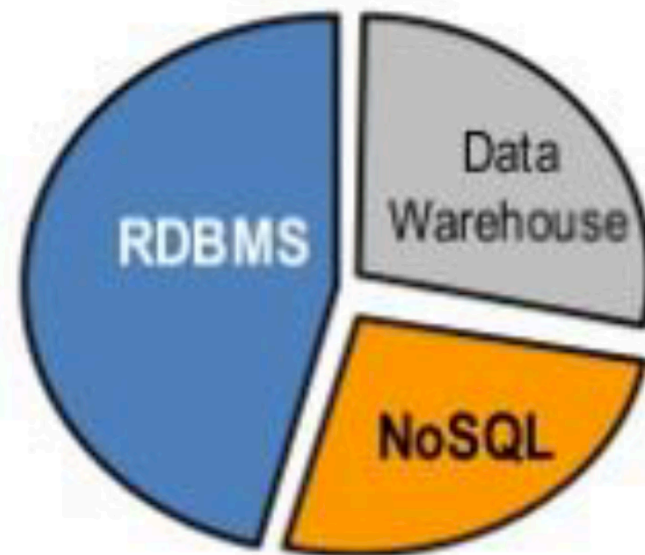
Répartition du SQL et du NoSQL au fil des années



1985-1995



1995-2010

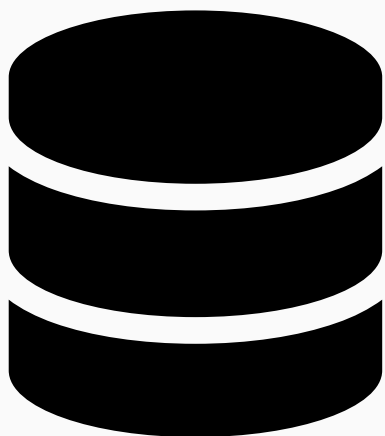


2010-Now



Bases de données et Stockage de Fichiers

Base de données



- Organisé
- Fonctionnalité de recherche, réplication...

Fichiers systèmes



- Moins organisé
- Simple, moins de fonctionnalité ajoutés



Qu'est-ce le calcul parallèle ?

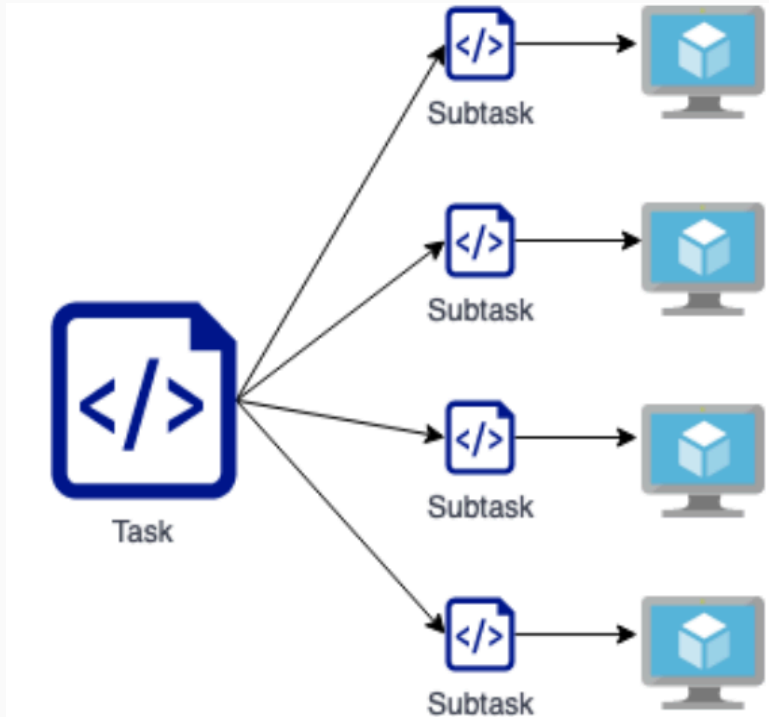
L'idée du calcul parallèle

Base des outils modernes de traitement des données

- Mémoire
- Puissance de traitement

Idée

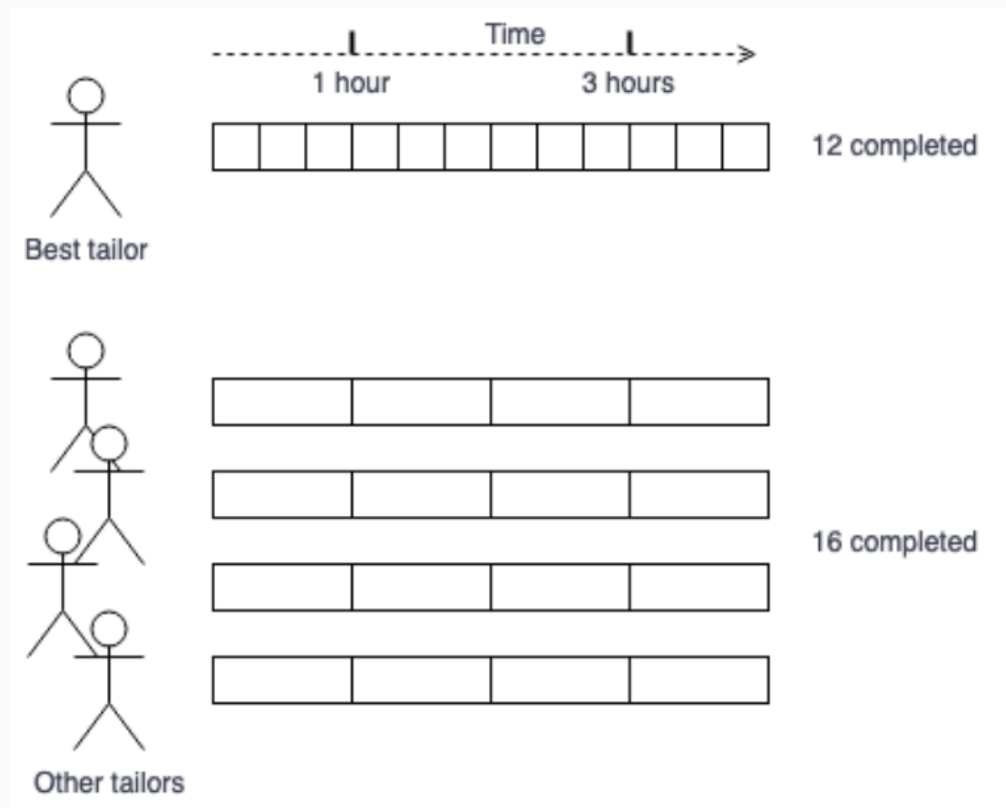
- Diviser la tâche en sous-tâches
- Répartir les sous-tâches sur plusieurs ordinateurs
- Travailler ensemble pour terminer la tâche





Qu'est-ce le calcul parallèle ?

Exemple : Un atelier de couture



Faire fonctionner un tailleur

But : 100 T-shirts construites

- Les meilleurs tailleurs font un t-shirt / 20min
- Les autres peuvent faire 1 t-shirt / 1h

Plusieurs tailleurs qui travaillent ensemble
> Meilleur tailleur.



Qu'est-ce le calcul parallèle ?

Avantages d'un calcul distribué

- Puissance de calcul
- Mémoire : Partitionnement d'un ensemble de données (dataset)



Qu'est-ce le calcul parallèle ?

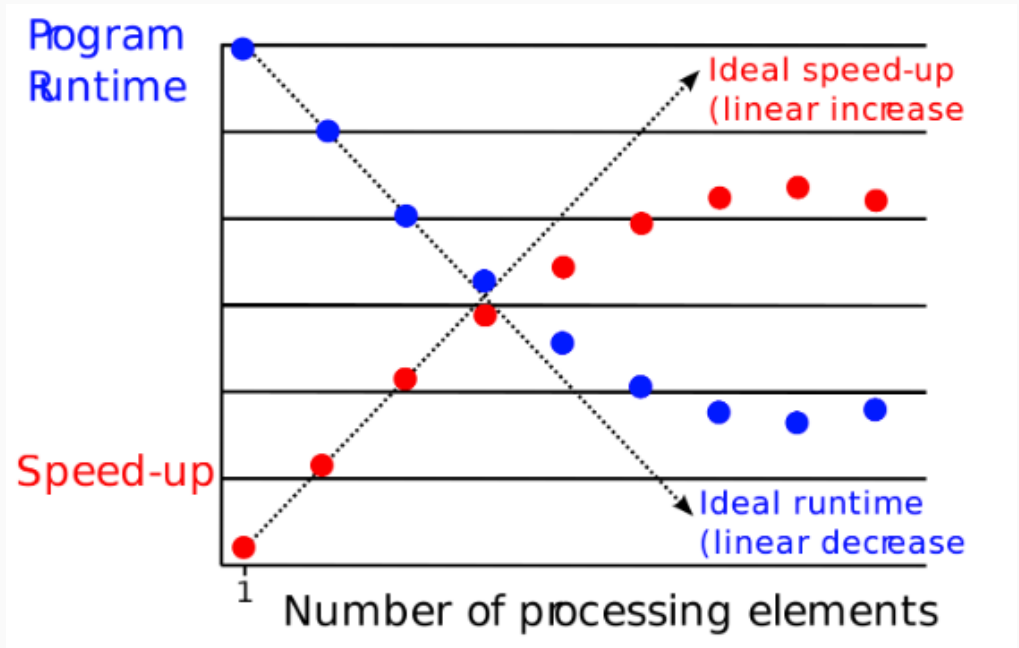
Risques d'un calcul distribué

Frais future lié à la communication

Base des outils modernes de traitement des données

- Les tâches doivent être volumineux
- Besoin de plusieurs process/threads

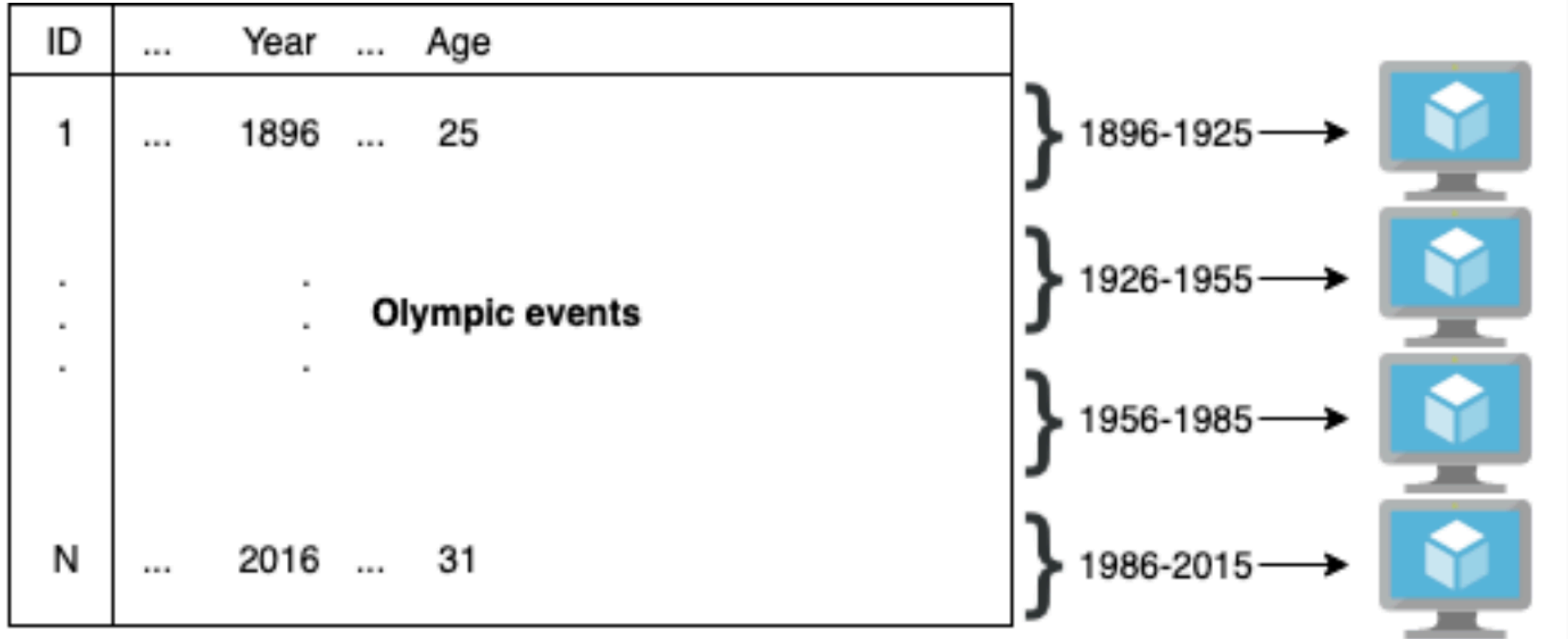
Ralentissement Parallèles





Qu'est-ce le calcul parallèle ?

Exemple : Un dataset des Jeux Olympiques





Qu'est-ce le calcul parallèle ?

Exemple en code (Bas niveau)

```
1  from multiprocessing import Pool python
2
3  def take_mean_age(year_and_group) -> pd.DataFrame:
4      year, group = year_and_group
5      return pd.DataFrame({"Age": group["Age"].mean()}, index=[year])
6
7  with Pool(4) as p:
8      results = p.map(take_mean_age, athlete_events.groupby("Year"))
9
10 result_df = pd.concat(results)
```



Qu'est-ce le calcul parallèle ?

Exemple en code (Haut Niveau)

```
1 import dask.dataframe as dd
2 # Nous partitionons le dataframe en 4
3 athlete_events_dask = dd.from_pandas(athlete_events, npartitions = 4)
4
5 # Nous lançons le calcul parallèle sur chaque partition
6 result_df = athlete_events_dask.groupby('Year').Age.mean().compute()
```

python



Activité : Why parallel computing?

You've seen the benefits of parallel computing. However, you've also seen it's not the silver bullet to fix all problems related to computing.

Which of these statements is not correct?

1. Parallel computing can be used to speed up any task.
2. Parallel computing can optimize the use of multiple processing units.
3. Parallel computing can optimize the use of memory between several machines.



Activité : Why parallel computing?

You've seen the benefits of parallel computing. However, you've also seen it's not the silver bullet to fix all problems related to computing.

Which of these statements is not correct?

1. Parallel computing can be used to speed up any task.
2. Parallel computing can optimize the use of multiple processing units.
3. Parallel computing can optimize the use of memory between several machines.

Réponse : 1



Activitié : From task to subtasks

```
# Function to apply a function over multiple cores
```

```
@print_timing
```

```
def parallel_apply(apply_func, groups, nb_cores):
```

```
    with Pool(nb_cores) as p:
```

```
        results = p.map(apply_func, groups)
```

```
    return pd.concat(results)
```

```
# Parallel apply using 1 core
```

```
parallel_apply(take_mean_age, athlete_events.groupby('Year'), ____)
```

```
# Parallel apply using 2 cores
```

```
parallel_apply(take_mean_age, athlete_events.groupby('Year'), ____)
```

```
# Parallel apply using 4 cores
```

```
parallel_apply(take_mean_age, athlete_events.groupby('Year'), ____)
```



Activitié : From task to subtasks (corrigé)

```
# Function to apply a function over multiple cores
```

```
@print_timing
```

```
def parallel_apply(apply_func, groups, nb_cores):
```

```
    with Pool(nb_cores) as p:
```

```
        results = p.map(apply_func, groups)
```

```
    return pd.concat(results)
```

```
# Parallel apply using 1 core
```

```
parallel_apply(take_mean_age, athlete_events.groupby('Year'), 1)
```

```
# Parallel apply using 2 cores
```

```
parallel_apply(take_mean_age, athlete_events.groupby('Year'), 2)
```

```
# Parallel apply using 4 cores
```

```
parallel_apply(take_mean_age, athlete_events.groupby('Year'), 4)
```



Activité : Using a dataframe

1. Print out the mean age for each Year. Remember dask uses lazy evaluation.
2. Create 4 partitions of the athletes_events DataFrame using dd.from_pandas().

```
1 import dask.dataframe as dd
2
3 # Set the number of partitions
4 athlete_events_dask = dd.from_pandas(athlete_events, __=__)
5
6 # Calculate the mean Age per Year
7 print(athlete_events_dask.groupby('Year').Age.___().___())
```

python



Activité : Using a dataframe (corrigé)

1. Print out the mean age for each Year. Remember dask uses lazy evaluation.
2. Create 4 partitions of the athletes_events DataFrame using `dd.from_pandas()`.

```
1 import dask.dataframe as dd
2
3 # Set the number of partitions
4 athlete_events_dask = dd.from_pandas(athlete_events, npartitions=4)
5
6 # Calculate the mean Age per Year
7 print(athlete_events_dask.groupby('Year').Age.mean().compute())
```

python



Historique : La naissance du Cloud

Lors du Black Friday en 2002, Amazon ne souhaite plus voir son site tomber en panne suite à la charge de travail lié au connexion.

- Il ont alors commandé plusieurs PC pour faire face à la demande.
- Sauf que, les PC ne sont pas utilisés sur le reste de l'année.



Historique : La naissance du Cloud

Lors du Black Friday en 2002, Amazon ne souhaite plus voir son site tomber en panne suite à la charge de travail lié au connexion.

- Il ont alors commandé plusieurs PC pour faire face à la demande.
- Sauf que, les PC ne sont pas utilisés sur le reste de l'année.

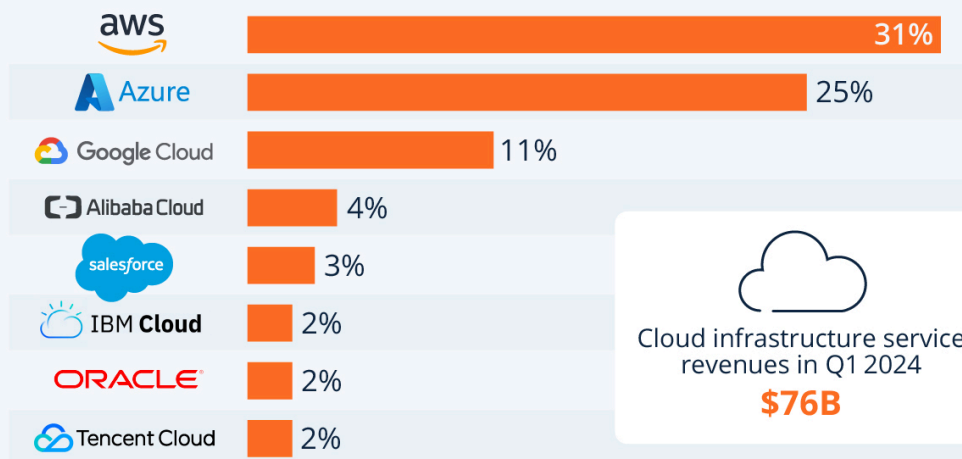
Solution : Louer ces même PC au entreprises, d'où la naissance du Cloud Computing.

Attention, j'ai pris un très gros raccourci pour illustrer la naissance du cloud. AWS n'est pas vraiment le premier, mais a fortement contribué en ce sens.

Les big players actuels

Amazon Maintains Cloud Lead as Microsoft Edges Closer

Worldwide market share of leading cloud infrastructure service providers in Q1 2024*

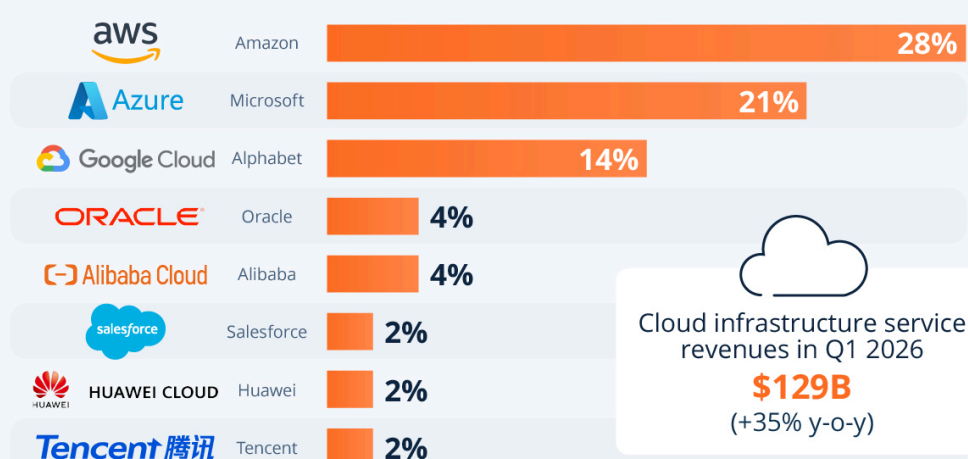


* Includes platform as a service (PaaS) and infrastructure as a service (IaaS) as well as hosted private cloud services

Source: Synergy Research Group

Big Three Hold Dominant Lead in Accelerating Cloud Market

Worldwide market share of leading cloud infrastructure service providers in Q1 2026*



* Includes platform as a service (PaaS) and infrastructure as a service (IaaS) as well as hosted private cloud services

Source: Synergy Research Group



statista



statista



Un théorème fondamentale CAP

Un des concept fondamentale dans le monde du Big Data au niveau de l'architecture est le théorème CAP, autrement dit Brewer :

| Théorème de Brewer

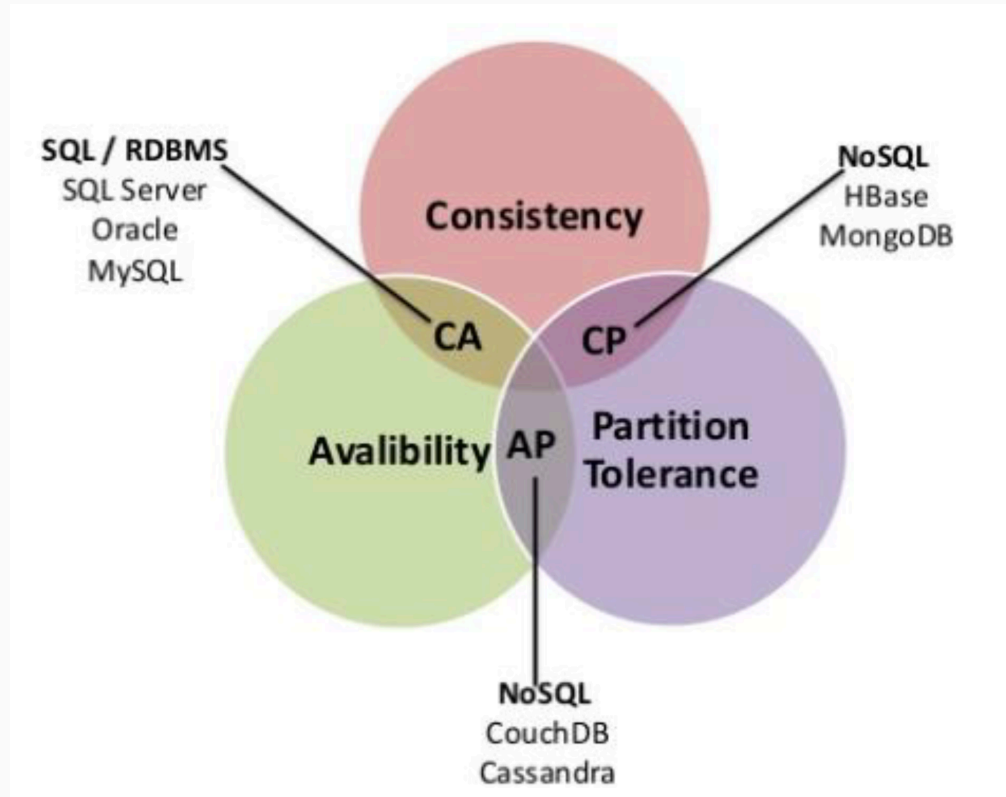
Il est impossible pour un système d'information de calcul distribué de garantir les trois contraintes suivantes :

- **Cohérence** (Consistency) : Tout noeud du système voient exactement les même données au même moment.
- **Disponibilité** (Avability) : Tout requête reçu a une garanti de réponse
- **Tolérance au partitionnement** : (Partition Tolerance) : Aucune panne moins importante qu'une coupure de courant / coupure totale de réseau doit compromettre le fonctionnement du système d'information. (Chaque noeud doit pouvoir fonctionner en indépendance des autres).



Un théorème fondamentale CAP

Nous avons donc les éléments suivants possible pour le théorème CAP



- **CP** : Consistency + Partition Tolerance
- **CA** : Consistency + Availability
- **AP** : Availability + Partition Tolerance



Activity : Applying CAP theorems

Suppose that you want to create a Twitter-like application and you are thinking about its architecture.

- Your application should be able to send, and pull tweets from other, and be able to follow other accounts in real time.
- Data are consumed right away and it is unlikely that users look for old tweets (unless they want to break someone's career).
- **Which two constraints of the CAP theorem should you meet for your application?**



En revue le théorème de Brewer

CP : Consistency + Partition tolerance

Nous allons privilégier la consistance des données au détriment de la disponibilité

i | Exemple : Transaction financière

Les banques ou les marchés devront prioriser la consistance des données, surtout sur des grosse volumétries transactionnel.

- *Si le système détecte une partition, il va bloquer la transaction afin de refléter la réalité du solde disponible.*

i | Exemple : Système d'inventaire

Les plateformes e-commerces doivent s'assurer de ne pas faire plus de vente d'inventaire que de disponible.

- *Lorsque nous avons une partition, elle préfère refuser la commande de client plutôt qu'à honorer la commande dont l'inventaire n'est pas disponible*



En revue le théorème de Brewer

AP : Availability + Partition Tolerance

Nous allons privilégier la disponibilité au détriment de la consistance

i | Exemple : Plateformes de réseaux sociaux

Des systèmes tels que Facebook ou Twitter peuvent privilégier la disponibilité afin de garantir que les utilisateurs puissent toujours publier, commenter et envoyer des messages, même si certaines données sont légèrement désynchronisées.

i | Exemple : Réseaux de diffusion de contenu (CDN) :

Un CDN doit toujours être disponible pour diffuser rapidement du contenu, même si certains nœuds ne sont pas encore synchronisés avec la dernière version des données.



En revue le théorème de Brewer

CA : Consistency + Availability

Nous allons privilégier la consistance des données et la disponibilité, en supposant qu'il n'y a pas de partition réseau

i | Exemple : Base de données relationnelle centralisée

Une base de données relationnelle exécutée sur un seul serveur peut fournir des lectures cohérentes et répondre aux requêtes tant que le serveur fonctionne.

- *Toutes les opérations sont effectuées sur une même source de vérité.*
- *Les lectures observent immédiatement les écritures validées.*
- *Il n'existe pas de partition entre plusieurs réplicas à gérer.*

i | Exemple : Application en mémoire sur un nœud unique

Une application qui conserve son état sur une seule machine peut être à la fois cohérente et disponible, à condition que cette machine reste accessible.

- *Les utilisateurs lisent le même état partagé.*
- *Les mises à jour sont immédiatement visibles.*
- *En cas de panne du nœud, la disponibilité est perdue.*

Pour aller plus loin, théorème PACELC prenant en compte la latence.



Activity

Question: Which of the following statements accurately describes the CAP theorem in the context of distributed systems?

1. The CAP theorem states that a distributed system can achieve Consistency, Availability, and Partition tolerance simultaneously without trade-offs.
2. According to the CAP theorem, partition tolerance is irrelevant in modern distributed systems because networks are always reliable.
3. The CAP theorem states that in a distributed data store, it is impossible to guarantee all three of the following simultaneously: Consistency, Availability, and Partition tolerance. You can only have two of the three at any given time.
4. The CAP theorem emphasizes the importance of using a single database to avoid any trade-offs between consistency, availability, and partition tolerance.



Activity

Question: Which of the following statements accurately describes the CAP theorem in the context of distributed systems?

1. The CAP theorem states that a distributed system can achieve Consistency, Availability, and Partition tolerance simultaneously without trade-offs.
2. According to the CAP theorem, partition tolerance is irrelevant in modern distributed systems because networks are always reliable.
3. The CAP theorem states that in a distributed data store, it is impossible to guarantee all three of the following simultaneously: Consistency, Availability, and Partition tolerance. You can only have two of the three at any given time.
4. The CAP theorem emphasizes the importance of using a single database to avoid any trade-offs between consistency, availability, and partition tolerance.



Notion de Scalabilité

Le monde du Big Data possède 3 familles d'infrastructure à savoir : Un Serveur Centralisé, une machine virtuel dans un serveur (Scale-up), un cluster (Scale-Out)

Notion de Serveur Centralisé

i | Définition : **Serveur Centralisé**

Principe de faire fonctionner ce système sur un seul serveur. Nous avons donc une notion d'un Serveur avec plusieurs clients.

Avantages

- Management simplifié
- Consistance
- Performance relative élevé
(Remplacement des composants/parc)

Désavantages

- Bottleneck
- Flexibilité limités
- Risque élevé de saturation des ressources



Notion de Scalabilité

Notion de Scalabilité Verticale

i | Définition : **Scale-up**

Principe de faire fonctionner ce serveur avec des ressources limitées. Nous pouvons ajouter des ressources supplémentaire pour répondre à un pic d'activité élevé.

Avantages

- Coûts maîtrisées
- Simplicité à mettre en œuvre
- Performance relatives élevé.
(Remplacement des composants)

Désavantages

- Âge de l'hardware
- Latence (Doubler la performance d'une machine ne permet pas de doubler la performance du système)
- Risques maintenance (Notion de downtime, le temps d'une migration vers un nouveau serveurs)



Notion de Scalabilité

Notion de Scalabilité à l'Horizontale.

i | Définition : **Scale-Out**

Principe de faire fonctionner ce serveur sur plusieurs serveur de manière distribués. En cas de pic de consommateurs, nous ajoutons un serveur supplémentaire.

Avantages

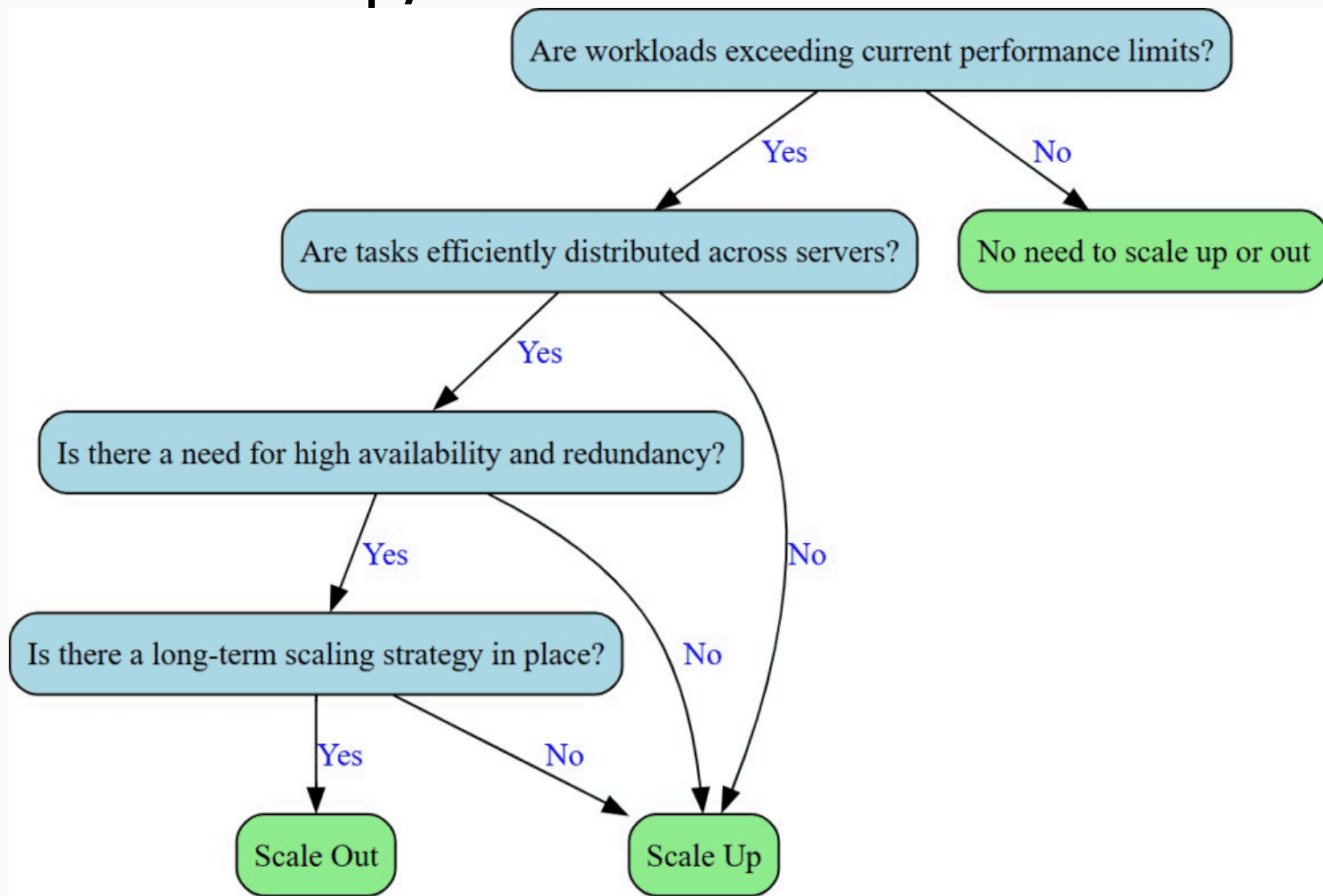
- Adaptation à la demande
- Coût maîtrisés et anticipable.
- Transition de serveurs nouvelle générations simplifié (Last New In, First Old Out)

Désavantages

- Espace disque limités
- Coût opérationnels élevés
- Coût de départ élevé dès la conception.



Quand faire du Scale-up / Scale-Out ?





Les 3 V principaux du Big Data

i | Définition : **Volume:**

Désigne une certaine quantité de donnée stocké au sein de l'architecture devant être traité.

Le nom de Big-Data, est lui-même associé à une grande quantité de données.

Pour déterminer la valeur des données, la taille des données joue un rôle crucial.

- Si le volume des données est très important, on parle alors de « Big Data ».
- En d'autres termes, la question de savoir si des données particulières peuvent être considérées comme des Big Data ou non dépend du volume des données.

Par conséquent, lorsqu'on traite des Big Data, il est nécessaire de tenir compte d'une caractéristique, le « volume »

i | Exemple :

En 2016, le trafic mobile mondial était estimé à 6,2 exaoctets (6,2 milliards de Go) par mois.

En outre, d'ici 2020, nous disposerons de près de 40 000 exaoctets de données.



Les 3 V principaux du Big Data

i | Définition : **Vitesse**

: Désigne la vitesse de calcul d'un traitement de donnée. qu'elle soit générée ou traitée.

La vitesse fait référence à la vitesse d'accumulation des données.

Dans le cadre du Big Data, les données circulent à partir de sources telles que les machines, les réseaux, les médias sociaux, les téléphones portables, etc.

Il s'agit d'un flux massif et continu de données. Il existe un flux massif et continu de données. Cela détermine le potentiel des données, c'est-à-dire la vitesse à laquelle les données sont générées et traitées pour répondre aux demandes.

L'échantillonnage des données peut aider à traiter la question de la "vitesse".

i | Exemple :

Plus de 3,5 milliards de recherches sont effectuées chaque jour sur Google. De même, le nombre d'utilisateurs de Facebook augmente de 22 % (environ) d'année en année.



Les 3 V principaux du Big Data

i | Définition : **Variété**

Désigne les différents format/types de représentation d'une donnée (Fichier plats tel que XML, JSON..., Audio, Vidéo, Stream...) On dénote principalement les données **Structurés, Semi-structurés, Non-Structurés**

i | Définition : **Données Structurés (Rappel) :**

Ces données sont essentiellement des données organisées. Il s'agit généralement de données dont la longueur et le format ont été définis. **Autrement dit, ce sont les données relationnels (ou bases de données relationnel).**

Exemple : Les bases de données SQL, un fichier CSV.



Les 3 V principaux du Big Data

i | Définition : **Données Semi-Structurés (Rappel)**

Une donnée semi-structurée est une donnée qui s'apparente à une donnée structurée, mais qui va pouvoir assurer une certaine flexibilité des champs. Il s'agit généralement d'une forme de données qui ne se conforment pas à la structure formelle des données.

Les fichiers journaux sont des exemples de ce type de données :

- On va avoir des hiérarchies qui seront variable.
- Des métadonnées, voir des balises.
- Des formats variées (Json, XML, YAML, ..)
- La facilité d'extension, ajout des information sans modifier de base. (Extension de schéma)
- Assurer une complexité variable (Liste chaîné imbriqué dans une autre liste par exemple).

```
1 {"key": "value"}
```

```
} json
```



Les 3 V principaux du Big Data

i | Définition : **Données Non-Structurés (Rappel)**

Ces données font essentiellement référence à des données non organisées. Il s'agit généralement de données qui ne s'intègrent pas parfaitement dans la structure traditionnelle en lignes et en colonnes de la base de données relationnelle.

Les textes, les images, les vidéos, etc. sont des exemples de données non structurées qui ne peuvent pas être stockées sous forme de lignes et de colonnes.

On dit également qu'elle ne comportent pas de schéma.



Exemple des 3 V en application médicale :

- Les outils informatiques utilisés proviennent des objets connectés (IoT). Elle génèrent de la donnée et admettent donc un certain volume qui doit être stocké et traité.
 - C'est la propriété d'un **Volume de données**.
- Elle peut également avoir des sources de données qui ne proviennent pas de l'IoT, cela peut représenter des dossiers médicaux du patient, le dossier administratif, les publications de veille technologique...
 - Nous avons donc ici la propriété de la **Variété**.
- Les données peuvent avoir une importance sur son délai de traitement, l'exemple le plus notable sont les cas d'urgences, ce qui met en avant la propriété de la **Vélocité**.



4 << V >> additionnels

Au cours de la dernière décennie, d'autres V, non majeur, ont fait leur apparition :

i | Définition : **Valeur**

Nous parlons de valeur lorsque cette donnée est utilisé pour répondre à une problématique de Data Science et/ou de Big Data

Les données brutes n'ont aucune valeur pour les entreprises, il est donc important de créer de la valeur.

i | Exemple :

Premier exemple : *Nous récupérons les données d'un athlète de Tir à l'arc, à l'aide de l'Apple Watch. Nous réutilisons ces données afin de générer un rapport de performance sur l'athlète, mentionnant les axes d'améliorations.*

Deuxième exemple : *Nous récupérons des données de deux pongistes en temps réelle afin de pouvoir générer des commentaires textuelle.*



4 << V >> additionnels

i | Définition : **Variabilité**

Nous parlons de variabilité pour les différences sur la fréquence d'actualisation, disponibilité de ces données.

- Nous pouvons donc poser des question tel que :
 - Quel vitesse ou dans quel mesure la structure de vos données évolue-t-elle ?
 - À quelle fréquence la signification ou la forme de vos données change-t-elle ?

i | Exemple :

Si vous mangez la même glace tous les jours et que son goût change constamment.



4 << V >> additionnels

i | Définition : **Véracité**

Une incohérence et une incertitude sur ces données.

- C'est-à-dire que les données disponibles peuvent parfois être désordonnées et qu'il est difficile d'en contrôler la qualité et l'exactitude.
- Le Big Data est également variable en raison de la multitude de dimensions de données résultant de multiples types et sources de données disparates.

i | Exemple :

Des données en vrac peuvent créer de la confusion, tandis qu'une quantité moindre de données peut transmettre des informations incomplètes.



4 << V >> additionnels

Visualisation

Aujourd'hui, la visualisation des données joue un rôle majeur dans le Big Data :

- **Elle aide à la prise de décision à partir des données traités en amont**

Nous les retrouvons principalement sous la forme de **tableau de bords**.



Apparté : Les langages

	Java	Scala	Python
Avantages	- Majorités des composant compatible JVM - Facilité de recrutements profils junior	- Les composants sont basé sur JVM - Programmation Fonctionnel (Calcul parallèle) - Facilement maintenable - Data Engineer expérimentés	- Plus de librairies Machine learning - Data Scientist expérimentés - Haute productivité
Désavantages	- Pas de productivité - Difficile de trouver des profils expérimentés DE et DS	- Rare profils orienté FP - Profils coûteuses du à la rareté	- Coûteux à la maintenance - Lent

02

Panorama Eco-système Hadoop (J1-P2)





Qu'est-ce Hadoop ?

i | Définition : **Hadoop**

Hadoop est un Framework logiciel « Open-Source » afin de stocker de grande volumétrie de données et de les traiter sur des clusters à faible coût.

- Création d'Hadoop en 2005 par Doug Cutting et Mike Cafarella.
- L'origine du nom est le nom d'un jouet d'éléphant de l'enfant de Doug.
- **Attention : Hadoop est plus précisément une collection de projets Open Source !**





Déplacer le traitement vers les données.

Initialement, les données devaient être traitées. Donc nous devons faire déplacer les données vers ces traitements.

Mais avec l'arrivée d'Hadoop, c'est le chemin inverse qui est réalisé.

⚠ | Mise en garde de Hadoop

Bien que Hadoop commence à ne plus être populaire, la majorité des entreprises ont investi dans cette stack, et sont pas encore prêtes à migrer vers le stockage S3.

De ce fait, les entreprises cherchent des DE qui sont capables de manipuler à la fois du HDFS et du S3. Ce cours couvre ce cas précis jusqu'à la généralisation de S3.



MapReduce et Algorithmme

Deux opérations sont réalisés : **Map** et **Reduce** (Bon trois finalement, en comptant le Shuffle)

i | Définition : **Map**

La fonction `map()` sert à découper le travail de récolte des données pertinentes au calcul de la requête utilisateur courante, suivant l'emplacement des fichiers en entrée.

i | Définition : **Reduce**

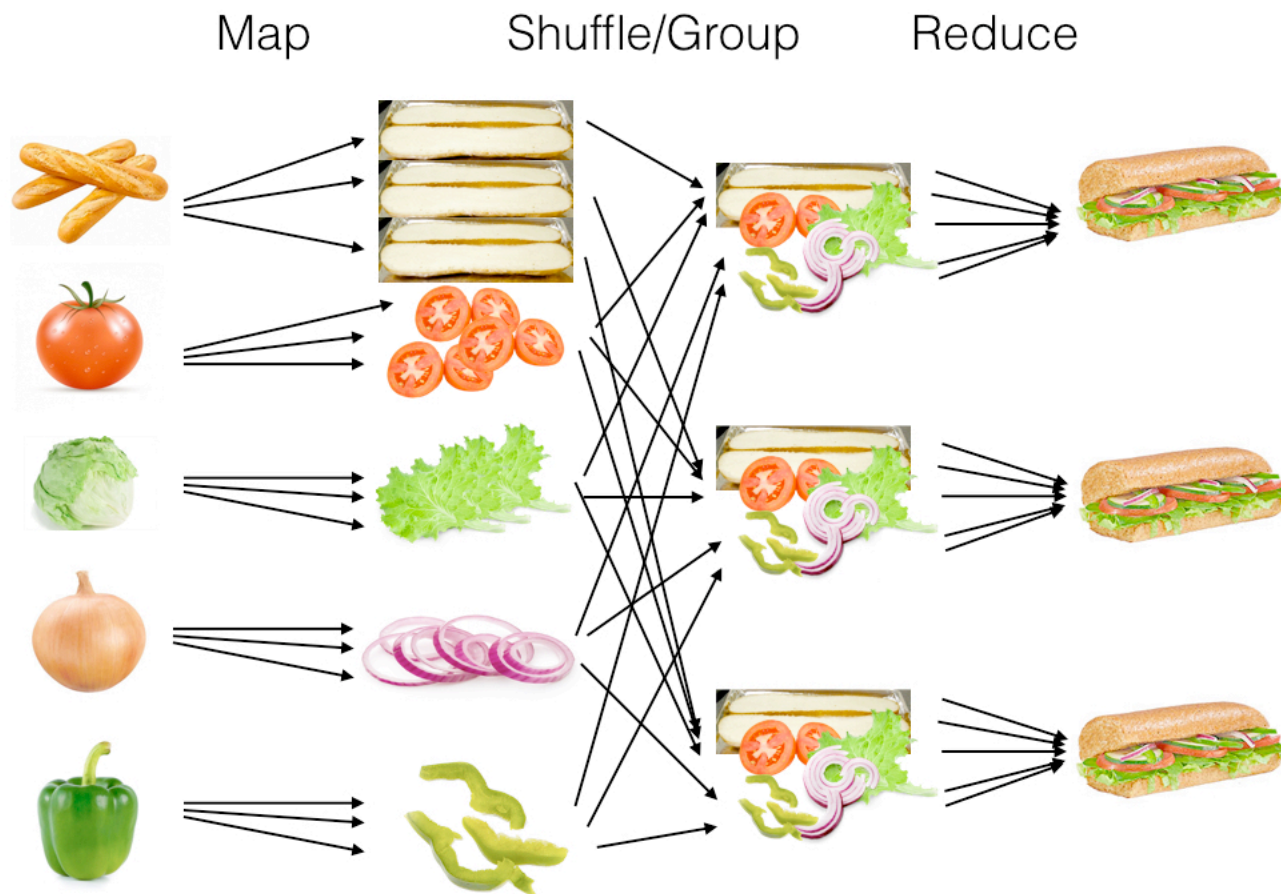
La fonction `reduce()` sert à réaliser le traitement de manière plus globale, sur les données en sorties des différentes fonctions `map()`.

i | Définition : **Shuffle**

Celle-ci s'occupera d'ordonner les sorties des mapper et d'optimiser leur transfert vers les reducer. Elle intervient entre le Map et le Reduce.



Exemple Analogique : MapReduce Subway





Exemple Analogique : MapReduce Subway



Steven Luscher

@steveluscher

Map/filter/reduce in a tweet:

map([, , ], cook)
=> [, , ,]

filter([, , ,], isVegetarian)
=> [, ,]

reduce([, ,], eat)
=> 



Scalabilité : Un point centrale

La Scalabilité sous Hadoop est un point essentiel. En effet, l'architecture Hadoop est basé sur ce qu'on appelle le **calcul distribué d'un cluster**. Ce qui permet d'optimiser le temps de calcul.

Les unités de traitement et de stockages sont peu coûteuses, on peut se permettre de monter en charge très facilement à moindre coût.



Fiabilité de l'architecture

L'ensemble des composants d'Hadoop est construit en garantissant l'hypothèse de la fiabilité de l'architecture.

En effet, un ordinateur est forcément confronté à une panne qui lui est inévitable. Nous devons donc nous attendre à ce qu'ils tombent en panne.



Nouvelle approche Big Data

Nous pouvons garder toutes les données que nous avons, et que nous pouvons prendre et analyser les données de plusieurs nouvelles manière.

Un des exemples est le style « **Schema on Read** » :

Je peux lire les données et créer le schéma au fur et à mesure de la lecture des données, sans passer des heures à recréer un schéma qui tente de faire correspondre à une donnée en particulier.

Nous pouvons donc nous permettre de garder les données à l'état brute, et les projeter au fur et à mesure de sa lecture.



Quatre composants.

Il existe 4 composants principale qui constitue le Framework Hadoop :

- Hadoop Common,
- Hadoop Distributed File Systems (HDFS)
- Hadoop MapReduce
- Hadoop Yarn

L'ensemble de ces composants communiquent entre eux via SSH ou HTTPS.



Hadoop Common

i | Définition : **Common**

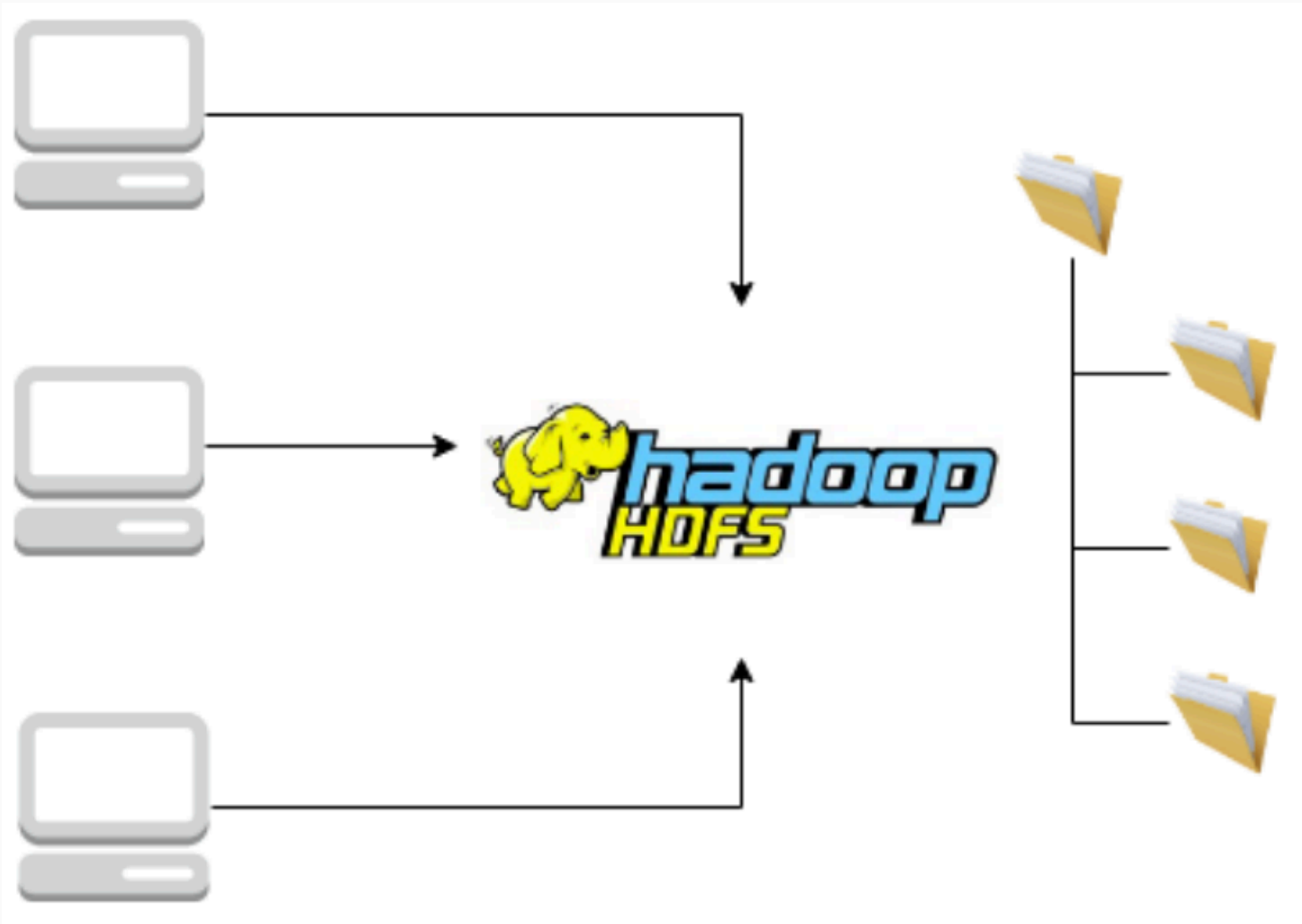
C'est l'ensemble des bibliothèques et utilitaires fondamentaux qui prennent en charge les autres modules d'Hadoop (comme HDFS, YARN, et MapReduce).

- Il comprend les fichiers de configuration, les outils d'administration et les composants nécessaires au bon fonctionnement d'un cluster Hadoop.

i | Définition : **HDFS**

HDFS (Hadoop Distributed File System) Un système de fichiers distribué d'Hadoop, conçu pour stocker de grandes quantités de données sur plusieurs nœuds d'un cluster constitués de machines modestes (**commodity servers**). Il divise les fichiers en blocs, les réplique pour assurer la tolérance aux pannes et permet un accès rapide aux données.

- Rédigé en Java pour le framework Hadoop.
- Gestion automatique de la réplication des données.
- Optimisé pour une lecture de fichier séquentielle.
- **La particularité principale est que les fichiers sont immuables !**





HDFS : Terminologie

NameNode Noeud principale possédant des métadonnées sur le système de fichier (nom de fichier, information dossier, bloc qui compose chaque fichier). Elle conserve les informations sur l'emplacement des fichiers sans stocker les données elle-même

DataNode Noeud de stockage dans HDFS, contient les blocs de données réels. Ils communiquent directement avec le NameNode de son état et fournissent les informations des blocs stockés (Responsable stockage, récupération et réplication données)

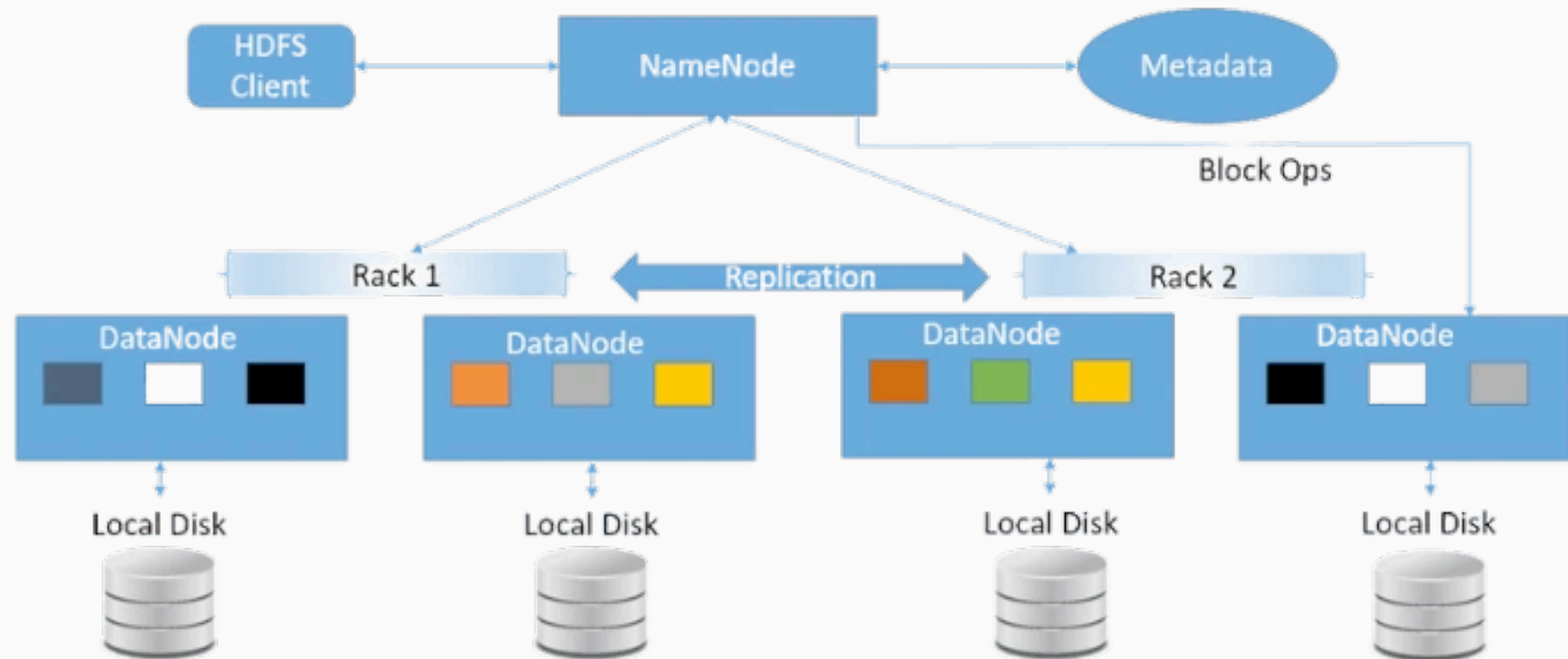
HDFS Client Interface Client/HDFS pour interagir avec les fichiers stockés dans HDFS. (Lecture/Ecriture au NameNode puis stockage aux DataNodes)

Au sens du théorème CAP : Elle respecte la Cohérence et la Tolérance au partitionnement des données.



HDFS : Terminologie

HDFS Architecture



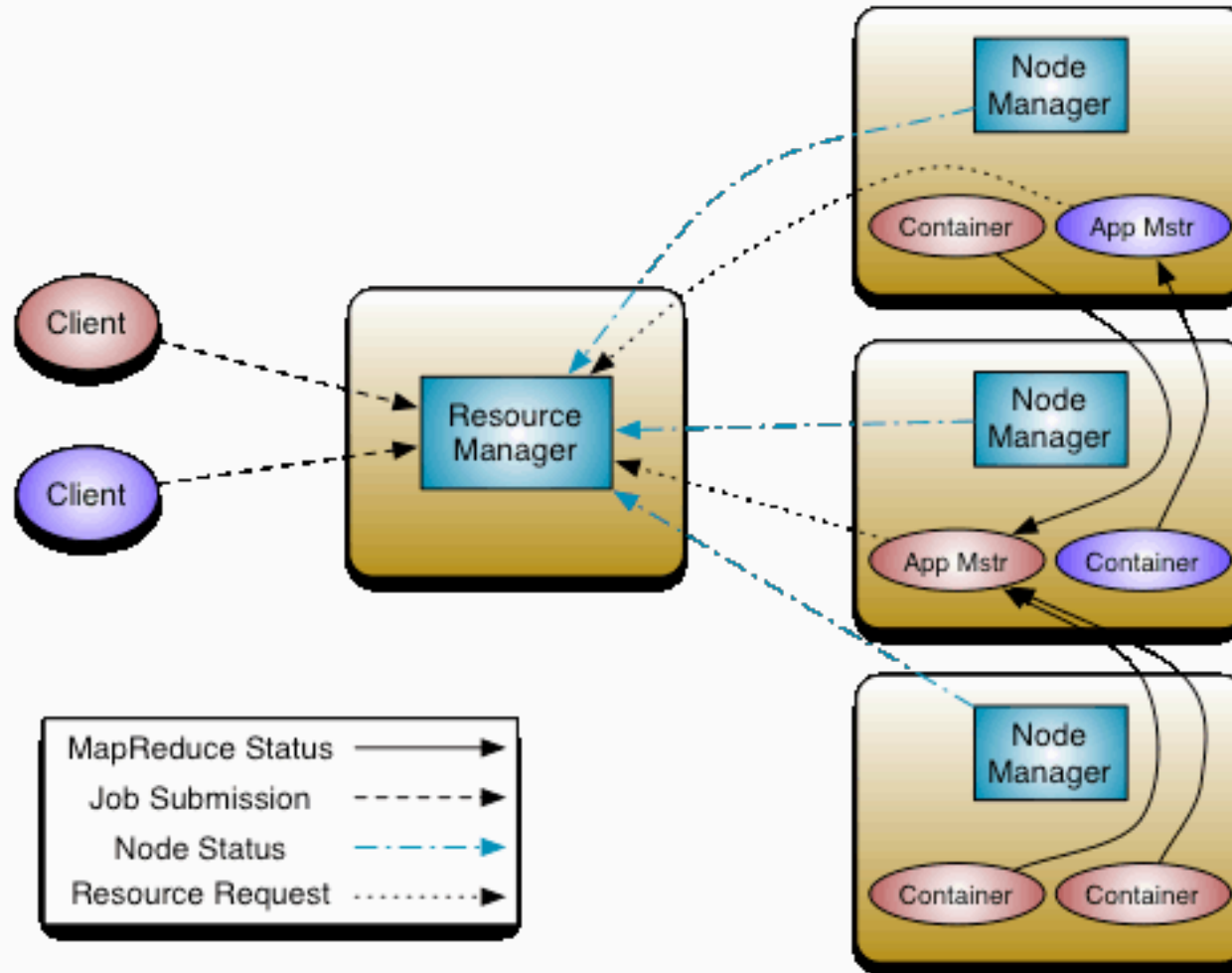
i | Définition : **YARN**

YARN (Yet Another Resource Negotiator) est un gestionnaire de ressources d'Hadoop qui coordonne le traitement et l'allocation de ressources sur un cluster.

- Il divise la gestion des ressources et la planification des tâches, permettant aux applications d'utiliser Hadoop de manière plus flexible et efficace.

Il faut voir cela comme une version 2 du MapReduce, depuis la sortie de Hadoop V2

Hadoop Yarn





Yarn : Terminologie

NodeManager Gère les ressources (Mémoire, CPU, GPU...) alloués aux applications de ce noeud. Supervise l'utilisation des ressources, l'état des containers qui execute les tâches des applications et rends les compte à ResourceManager

Resource Manager Maître de YARN, gère toute les ressources du cluster Hadoop. Elle répartie les ressources entre les applications en fonction du besoin. Garantie l'utilisation optimale du cluster.

- Possède un **Scheduler** qui décide l'allocation des ressources
- Possède un **Application Manager** qui gère le cycle de vie des applications

Application Master Composant spécifique à chaque application qui coordonne les run de tâches. Demande à Resources Manager des ressources pour ses NodeManagers.



Yarn : Terminologie

Client Interface pour utilisateur qui va soumettre un travail à YARN.

- Obtient un ID Application pour superviser l'exécution de la tâche demandé.

Container Unité de ressources de base dans YARN. Contient une mémoire et un nombre de CPU fixé alloué pour le run d'une application. (Gestion dynamique des ressources)

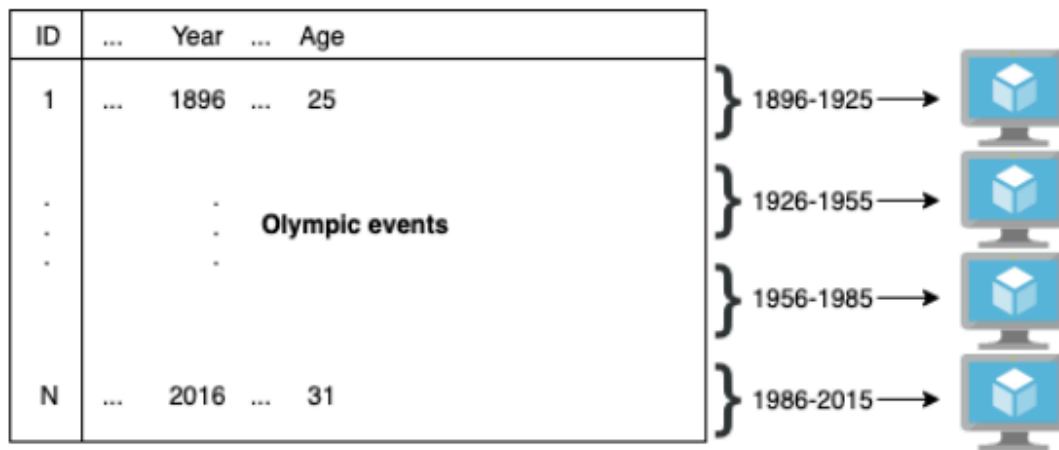


Hadoop MapReduce

i | Définition : **Map Reduce**

Map Reduce Un modèle de programmation, pour traiter et générer de grandes quantités de données en parallèle sur un cluster distribué. Il se compose de deux phases: Map, qui divise les données en tâches plus petites, et Reduce, qui agrège les résultats pour produire la sortie finale.

Hadoop MapReduce



- Base de données de type « **Data Warehouse** »
- Fonctionne sur Hadoop
- Langage de requête structuré : Hive SQL (Jointure + Tri)
- Initialement MapReduce, maintenant sur autres outils.
- Possibilité de rédiger des UDF en Java.
- Tables stockés directement dans HDFS.) en un RDD spark qui repr





Exemple : Hive

```
1 SELECT year, AVG(age)
2 FROM views.athlete_events
3 GROUP BY year
```

SQL

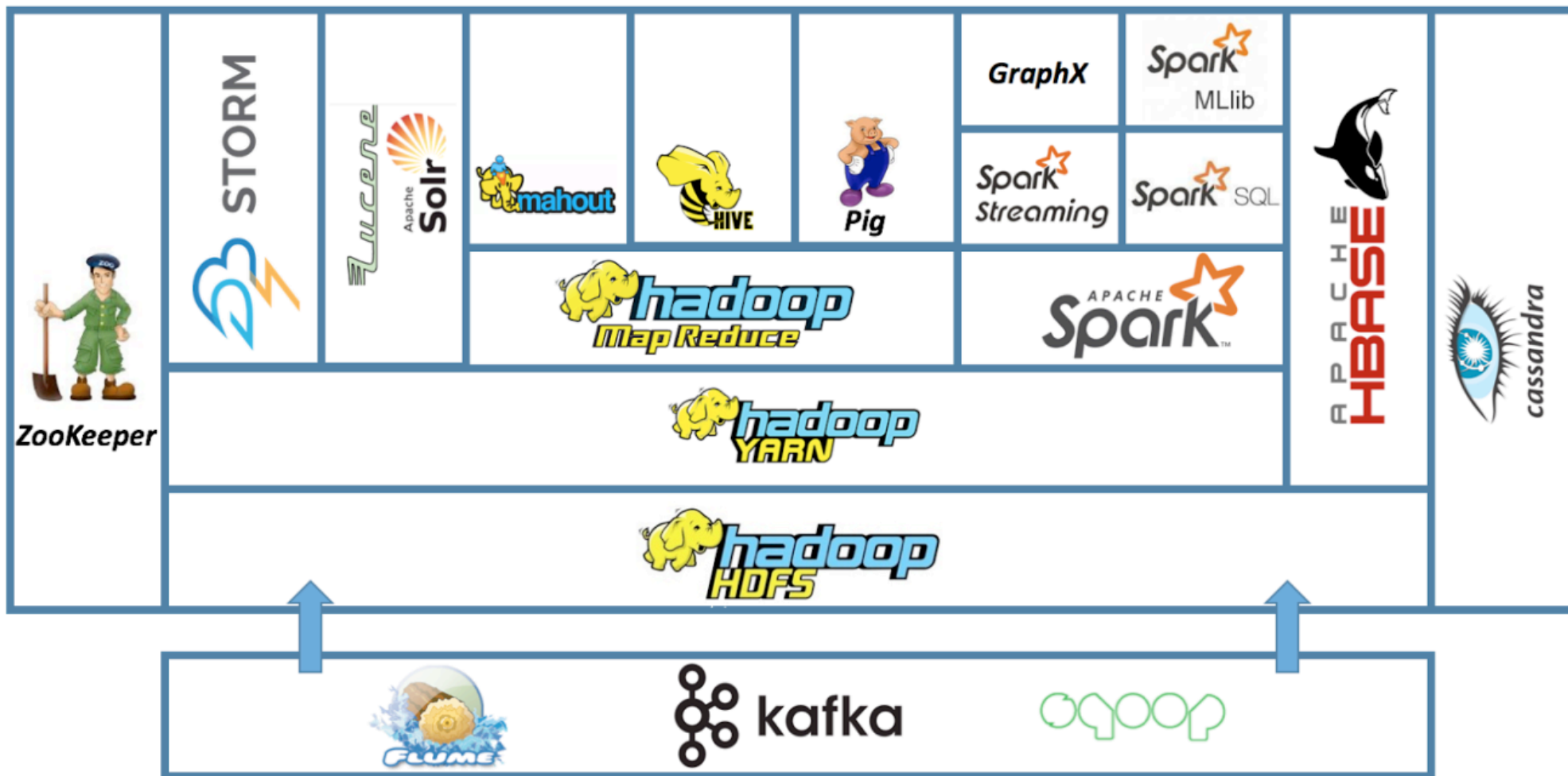


- Evite l'écriture sur les disques durs.
- Egalement maintenu par Apache Software Foundation

Cette librairie fera l'objet d'un chapitre entièrement dédié.



Ecosystème Hadoop





Activité : Spark, Hadoop and Hive

You've encountered quite a few open source projects in the previous slides. There's Hadoop, Hive, and PySpark. It's easy to get confused between these projects.

They have a few things in common: they are all currently maintained by the Apache Software Foundation, and they've all been used for massive parallel processing. Can you spot the differences?

Classify the cards to the corresponding software project.

1. Uses DataFrame abstraction
2. Scala or Python interface for the Spark framework
3. Initially used Hadoop MapReduce
4. Is built from the need to use structured queries for parallel processing
5. HDFS is part of it
6. MapReduce is part of it
7. Collection of open source packages for Big Data



Activité : Spark, Hadoop and Hive

Hadoop	Spark	Hive
5, 6, 7	1, 2	3, 4

Place au Travaux Pratique 1 !



Hadoop HDFS et Aujourd'hui

- Hadoop est malheureusement populaire aujourd'hui
 - Les entreprises qu'ils l'ont mis en place les cluster n'ont pas envie de le toucher
- D'autre entreprises en ont marre de le maintenir
 - Les compétences en Hadoop sont donc demandé pour réaliser des migration vers des **Object Storages** du cloud, ou son équivalent.

Nous avons donc une montée en puissance des stockage d'objet moderne:

- Cloud : AWS S3, Google Cloud Storage, Azure Blob Storage
- On premise :
 - Minio (A bannir définitivement, le projet est mort)
 - RustFS (A privilégier, en remplacement de Minio)
 - A considérer dans une architecture cloud-native : Ceph



Ressources additionnel

- Le futur logique de HDFS : L'object storage :
<https://resources.min.io/hdfs-migration/hadoop-hdfs-logical-successor>
- Plan de migration HDFS vers Minio / AWS S3 :
<https://resources.min.io/hdfs-migration/hadoop-migration-guide>
- Benchmark de performance d'un algorithme MapReduce entre Hadoop et Minio (dont le test WordCount du TP) :
<https://resources.min.io/hdfs-migration/minio-vs-hdfs-map-reduce>